

热身赛(01) T3 《染色方案》题解

简要题意

给定一棵 dep 层的完全二叉树。要从中选出一个包含根结点的连通块并染色，求有多少种不同的染色方案，答案对 998244353 取模。

多组数据。每组给出层数 dep ，以及一个长度为 dep 的 01 串 s ： s 是最后一层叶子数的二进制表示，高位用前导 0 补齐。下文把这个叶子数记作 L （即 $L = (s)_2$ ，由合法性保证 $1 \leq L \leq 2^{\text{dep}-1}$ ）。

数据范围

数据比例	T	dep	说明
30%	≤ 10	≤ 20	小数据，可暴力枚举
另 20%	≤ 10	$\leq 10^6$	额外保证树为满二叉树
100%	≤ 10	$\leq 10^6$	一般完全二叉树

综合约束： $1 \leq T \leq 10$ ， $1 \leq \text{dep} \leq 10^6$ 。题干特别提到“并不正常的时间限制”——这意味着需要线性级别的做法，一层层枚举必然超时。

这张表也暗示了拿部分分的顺序：先按第 2 关的公式直接递归，稳拿 30%；再特判满二叉树（答案就是第 3 关的 $f[\text{dep}]$ ），拿下另外 20%；最后处理一般情形。

第 1 关：题目到底在问什么？

目标：读懂“连通块”“包含根”这两个词，能手数出一棵小树的答案。

有一棵树，我们要给其中一些结点染色。染色的点必须满足两个条件：

- 必须包含最上面的根结点；
- 所有染色的点必须连成一片（不能东一个西一个，中间断开）。

问：有多少种不同的染色选择？答案可能很大，所以要对 998244353 取模。

先看一棵最小的树：只有 3 个结点——一个根，下面左边一个、右边一个。

图 1：3 个结点的树，一共 4 种染法（红 = 染色）

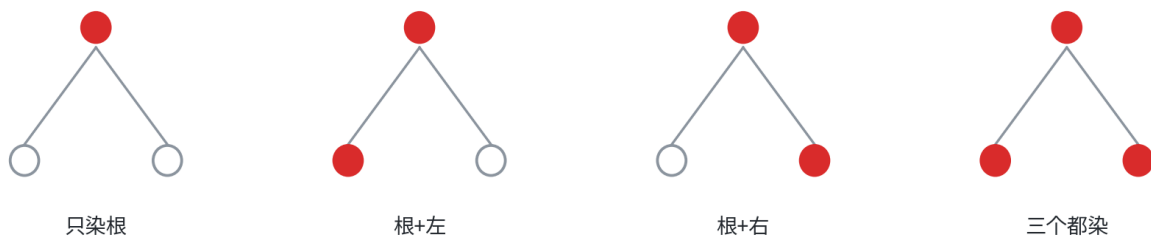


图 1：3 个结点的树，一共 4 种染法（红 = 染色）。

答案是 4 种：只染根；染根和左边；染根和右边；三个都染。

第 2 关：发现"乘法"的规律

目标：找到一条通用规则——一棵树的答案，可以由它左右两棵子树的答案"乘"出来。

为什么是 4 种？换个角度看：左边那个点可以染也可以不染（2 种），右边同理（2 种），左右互不打扰，乘法原理得 $2 \times 2 = 4$ 。

如果子树本身也是一棵大树呢？这时"左边"就不只是"染 / 不染"两种了。引入记号：

$g(u)$ = 以 u 为根的子树中，选一个包含 u 的连通块的方案数

设 u 的左右孩子分别为 l 、 r ：

- 左子树：不选（1 种），或选一个包含 l 的块（ $g(l)$ 种）→ 共 $g(l) + 1$ 种；
- 右子树：同理，共 $g(r) + 1$ 种。

$$g(u) = (g(l) + 1) \times (g(r) + 1)$$

约定：不存在的孩子视作 $g = 0$ ，于是它贡献的因子是 $0 + 1 = 1$ ，正好表示"这一侧什么都不选"。这条约定让公式对所有结点统一成立，叶子结点因此有 $g = (0+1)(0+1) = 1$ 。

验证：3 结点的树中 $g(l) = g(r) = 1$ ，代入得 $g(\text{根}) = (1+1) \times (1+1) = 4$ ✓

第 3 关：先搞定"整整齐齐"的树——满二叉树

目标：对左右完全对称的满二叉树，把答案预先算好，以后查表就能用。

满二叉树就是**每一层都排得满满的**、左右一模一样的树（有些教材称之为"完美二叉树"；本文的"满"一律指这个意思）。对这样一棵树，"高度"和"层数"是同一个数。

图 2：满二叉树——每层都排满，左右完全对称

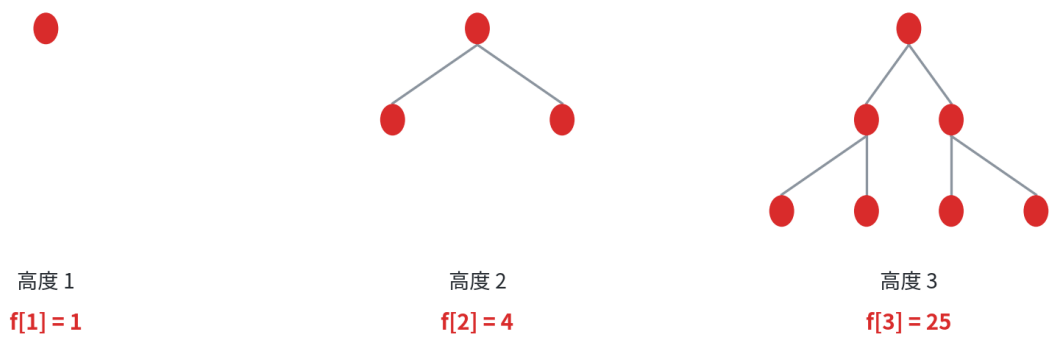


图 2：满二叉树，每层都排满，左右完全对称。

因为左右一模一样， $g(l) = g(r)$ 。记 $f[h]$ 为高度 h 的满二叉树的方案数，套用第 2 关公式：

$$f[h] = (f[h-1] + 1) \times (f[h-1] + 1) = (f[h-1] + 1)^2, \quad f[0] = 0$$

高度 h	怎么算的	$f[h]$
0	空树（约定值）	0
1	只有根，就 1 种	1
2	$(1+1)^2$	4
3	$(4+1)^2$	25
4	$(25+1)^2$	676
5	$(676+1)^2$	458329

为什么必须取模：第 5 层就到 45 万，第 6 层已经超过 2×10^{11} ，而 dep 最大是 10^6 。每一步乘法后都 $\% \text{MOD}$ ，数字就永远不会大到计算机存不下。

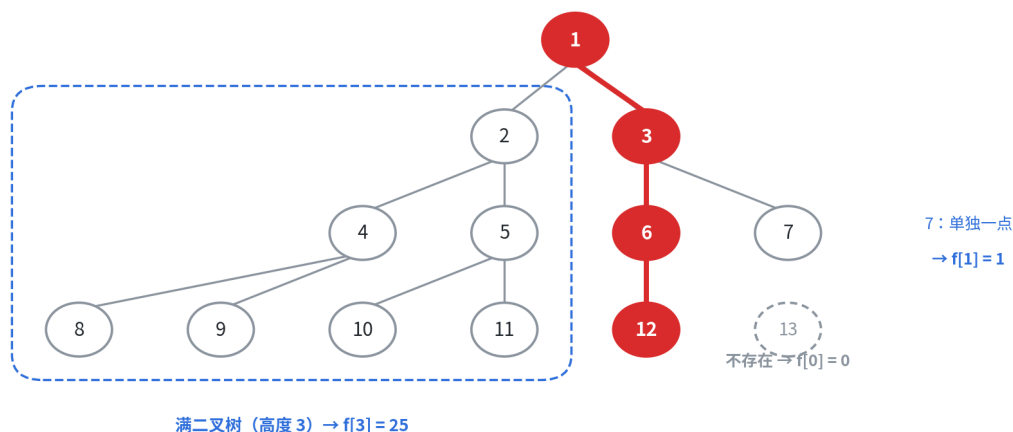
加餐（不要求掌握）：令 $a_h = f[h] + 1$ ，则 $a_h = a_{h-1}^2 + 1$ （ $a_1 = 2$ ），数列为 2, 5, 26, 677, 458330, ... 每往下长一层， a_h 的二进制位数大约翻一倍，所以 $f[h]$ 是"双重指数"级增长—— $h = 10$ 时它已经是一个 182 位的十进制数。这也是为什么只能打表 + $O(1)$ 查询，而不可能把它整个算出来。

第 4 关：一般情况——完全二叉树"哪里不整齐"？

目标：看清完全二叉树的结构，并严格说明路旁那棵子树到底有多高。

完全二叉树与满二叉树的唯一区别：最后一层可能没排满，而且是从左往右填，右边可能空着几个位置。

dep = 4, 最后一层 5 个叶子, 共 12 个结点 (红色 = 最右路径)



只有红色路径 $1 \rightarrow 3 \rightarrow 6 \rightarrow 12$ 上“不整齐”，路两旁挂着的都是整整齐齐的满二叉树

图 3: dep = 4、最后一层 5 个叶子 (共 12 个结点)。红色是“最右边那条路” $1 \rightarrow 3 \rightarrow 6 \rightarrow 12$ 。

关键发现 (本题的灵魂)：不整齐的地方只发生在“从根走到最右下角那个叶子”这一条路上，这条路两旁挂着的子树，一棵棵都是整整齐齐的满二叉树。

严格地说，路旁那棵树有多高？

把这条路上**从下往上**的结点依次记作 $v_0, v_1, \dots, v_{\text{dep}-1}$ ： v_0 是最后一个叶子 (深度 dep)， $v_{\text{dep}-1}$ 是根 (深度 1)。于是：

- v_j 的深度是 $\text{dep} - j$ ，它的子树一共有 $j + 1$ 层；
- 第 j 步 ($j = 0, 1, \dots, \text{dep}-2$) 做的，就是把 v_j 与它路旁的兄弟合并到父结点 v_{j+1} 上——一共 $\text{dep}-1$ 步，最后一步刚好合并到根。

那么第 j 步路旁那棵兄弟子树有多高？

- 若 v_j 是**右孩子**：它左边的位置一定全被填满了 (最后一层是从左往右填的)，所以它的**左兄弟**是高度 $j + 1$ 的**满二叉树**，贡献 $f[j + 1]$ ；
- 若 v_j 是**左孩子**：填充到它的子树就停了，它**右兄弟**的子树最后一层一个都没填上，所以右兄弟是高度 j 的满二叉树，贡献 $f[j]$ 。($j = 0$ 时右兄弟压根不存在，高度 0，贡献 $f[0] = 0$ ，与公式一致。)

第 j 步路旁子树的贡献 $k = f[j + K_j]$ ， $K_j = 1$ 表示 v_j 是右孩子，0 表示左孩子

易错提醒：不要记成“位是 1 $\rightarrow$ 兄弟完整”。完整的满二叉树永远长在**左边**，矮一层的永远长在**右边**；“位”决定的是**该看哪一边** (位 1 看左边、位 0 看右边)，而不是决定“兄弟是否完整”。

第 5 关：题目给的二进制串，其实是一张"地图"

目标：把输入串 s 翻译成"从最后一个叶子走向根，每一步是左孩子还是右孩子"。

第一步：给结点编号，找出最后一个叶子

从上到下、从左到右编号（堆式编号）：第 1 层是 1 号，第 2 层是 2、3 号，第 3 层是 4~7 号……前 $\text{dep}-1$ 层一共 $2^{\text{dep}-1} - 1$ 个结点，最后一层有 L 个，所以最后一个叶子的编号 K 为：

$$K = (2^{\text{dep}-1} - 1) + L = 2^{\text{dep}-1} + (L - 1)$$

第二步：把 s 减 1，就得到了"走路说明书"

上面这个变形是本题最漂亮的一步。因为 $L - 1 < 2^{\text{dep}-1}$ ，它恰好能塞进 $\text{dep}-1$ 个二进制位，于是：

K 的二进制 = 最高位的 1，后面接上 $(L - 1)$ 的 $\text{dep}-1$ 位表示。

换句话说：把输入的 s 减 1，剩下的低 $\text{dep}-1$ 位，就是"从最后一个叶子走向根"的说明书，第 j 位（从最低位数起）就是上一关的 K_j 。

$s = 0101$ （叶子数 $L = 5$ ）



$s - 1 = 0100$ （ $= L - 1 = 4$ ）



取 $s - 1$ 的低 $\text{dep}-1$ 位 = 100，前面补上最高位的 1 $\rightarrow K = 1100$ （二进制）= 12 —— 正是最后一个叶子的编号

图 4： $\text{dep} = 4$ 、 $L = 5$ 时， $s = 0101$ ，减 1 得 0100；取其低 3 位 100 便拼出 $K = 1100$ 。

对照图 3 一步一步看（ K_0 是最低位）：

第几步 j	位 K_j	含义	从 v_j 走到 v_{j+1}	路旁那棵子树	贡献 k
0	0	12 是左孩子	$12 \rightarrow 6$	看右边：13 不存在	$f[0] = 0$
1	0	6 是左孩子	$6 \rightarrow 3$	看右边：7，单独一个点	$f[1] = 1$
2	1	3 是右孩子	$3 \rightarrow 1$	看左边：2，高度 3 的满二叉树	$f[3] = 25$

最高位那一位不用管：它是 K 的"根标记"，永远落在根结点上，不产生兄弟。

第 6 关：一边往上走，一边把满子树"打包"进答案

目标：把路上遇到的每棵满子树，合并成一个总的方案。

这里的 `ans` 有明确的组合意义：**站在 v_j 上时，它就是 $g(v_j)$** ——以 v_j 为根的子树中，选一个包含 v_j 的连通块的方案数。

- **起点**：最后一个叶子自己就是一个连通块， $g = 1$ ，所以 $ans = 1$ ；
- **每往上一层**：父结点要把"当前这棵子树"和"路旁那棵满子树"连起来，套用第 2 关公式：

$$ans \leftarrow (ans + 1) \times (k + 1), \quad k = f[j + K_j]$$

这个 "+1" 和第 2 关意思相同——代表"这一整块都不要"。两块都不要时正好是"只染根"那 1 种，公式已经自动包含。

完整演算一遍 ($dep = 4$, $s = 0101$, 即 $L = 5$)

步骤	现在站在	路旁兄弟	k	算式	新的 ans
起点	$v_0 = 12$	—	—	$g(12) = 1$	1
第 0 步	$v_1 = 6$	不存在的 13	$f[0] = 0$	$(1+1) \times (0+1)$	2
第 1 步	$v_2 = 3$	单独一点 7	$f[1] = 1$	$(2+1) \times (1+1)$	6
第 2 步	$v_3 = 1$ (根)	满二叉树 2 (高度 3)	$f[3] = 25$	$(6+1) \times (25+1)$	182

最终答案是 **182**。全程只做了 3 次乘法。

交叉验算 (推荐你自己算一遍)：这棵树 12 个结点，逐层代第 2 关公式：

$g(8)=g(9)=g(10)=g(11)=g(12)=1$, $g(7)=1$; $g(4)=(1+1)(1+1)=4$, $g(5)=4$, $g(6)=(1+1)(0+1)=2$; $g(2)=(4+1)(4+1)=25$, $g(3)=(2+1)(1+1)=6$; **$g(1)=(25+1)(6+1)=182$** ✓ 与上面的结果一致。

最常见的错误：把 `ans` 初值写成 0。那会得到 $(0+1)(0+1)=1 \rightarrow 4 \rightarrow 130$ ，漏掉了"只染根"这一份方案。请记住：`ans` 的初值是 **1** (最后那个叶子自己)，不是 0。

第 7 关：写成代码

目标：把上面的思路翻译成 C++ 程序，并确认它不会超时、不会溢出。

变量清单

变量	它是什么	哪一关
<code>f[h]</code>	高度 h 的满二叉树的方案数（预先算好的查询表）， <code>f[0] = 0</code>	第 3 关
<code>dep</code>	树的层数	题目输入
<code>s</code>	最后一层叶子数 L 的 <code>dep</code> 位二进制串；减 1 之后就是路径说明书	第 5 关
<code>ans</code>	站在当前结点时的 g 值，初值 1	第 6 关
<code>Kj</code>	第 j 步的方向位：1 = 右孩子（看左兄弟），0 = 左孩子（看右兄弟）	第 5 关
<code>k</code>	路旁那棵满子树的方案数 <code>f[j + Kj]</code>	第 4 关

速度预估

- 预处理 `f[1] ... f[106]`：一次 10^6 次循环，毫秒级。
- 每组数据：串有多长就走多少步，`dep ≤ 106`、`T ≤ 10`，最多约 10^7 次操作。
- 内存：`f` 数组 10^6 个 `long long`，约 8 MB。
- 总复杂度 $O(10^6 + \sum dep)$ ，空间 $O(10^6)$ ，线性，稳过。

```
#include <bits/stdc++.h>
using namespace std;

typedef long long ll;
const int MAXN = 1000000 + 10; // 最多 100 万层，多留一点
const ll MOD = 998244353;      // 题目要求的模数

// f[h] = 高度 h 的满二叉树，包含树根的连通块方案数
ll f[MAXN];

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    // ---- 第 3 关：预先算好所有满二叉树的答案，f[0] = 0 表示空树 ----
    for (int h = 1; h < MAXN; ++h)
        f[h] = (f[h - 1] + 1) * (f[h - 1] + 1) % MOD;

    int T; // 有几组数据
    cin >> T;
    while (T--) {
        int dep;
        string s; // 长度 dep 的二进制串，数值为叶子数 L
        cin >> dep >> s;

        // ---- 第 5 关：s 减 1，剩下的低 dep-1 位就是"走回根"的说明书 ----
        //      K = 2^(dep-1) + (L - 1)，故 K 的低 dep-1 位 = (L-1) 的二进制
    }
```

```

int p = dep - 1;
while (p >= 0 && s[p] == '0') { s[p] = '1'; --p; } // 借位
if (p >= 0) s[p] = '0'; // 减 1 (L ≥ 1, 一定找得到 1)

// ---- 第 6 关: 从最低位往高位走, 边走边把路旁满子树合并进答案 ----
ll ans = 1; // 起点: 最后一个叶子自成一块, g = 1
for (int j = 0; j <= dep - 2; ++j) { // 一共 dep-1 步
    int Kj = (s[dep - 1 - j] == '1'); // 第 j 步的方向位
    ll k = f[j + Kj]; // 路旁兄弟子树的方案数
    ans = (ans + 1) % MOD * (k + 1) % MOD; // g(父) = (g(当前)+1)(g(兄弟)+1)
}
cout << ans << '\n';
}
return 0;
}

```

三个"为什么", 写代码前最好想明白

- **为什么每一步取模, 答案还是对的?** 因为模运算对加法和乘法是"保持"的: $(a+b) \bmod M = ((a \bmod M) + (b \bmod M)) \bmod M$, 乘法同理。所以"每算一步就取模"与"全部算完再取模"结果完全一样, 只是中间量不会爆掉。
- **为什么 long long 不会溢出?** 取模后 $ans, k < MOD$, 于是 $(ans+1)(k+1) \leq MOD^2 \approx 9.96 \times 10^{17}$, 而 long long 上限约 9.22×10^{18} , 还有近 10 倍余量, 安全。
- **为什么不用特判满二叉树?** 满树时 $L = 2^{\text{dep}-1}$, $s = 100\dots 0$, 减 1 后变成 $011\dots 1$, 每一位都是 1, 每一步都取 $f[j+1]$, 算出来正好是 $f[\text{dep}]$ 。统一处理即可, 代码更短。

自检样例 (写完代码先跑这几个)

dep	s	L	答案	说明
1	1	1	1	只有一个根, 边界
2	01	1	2	根 + 左孩子
2	10	2	4	满二叉树, = f[2]
3	001	1	6	最"瘦"的树
3	011	3	15	—
3	100	4	25	满二叉树, = f[3]
4	0001	1	35	—
4	0101	5	182	本文演算的例子
4	1000	8	676	满二叉树, = f[4]
5	00001	1	936	—

回顾：我们走了哪几步？

1. 手数小例子 → 3 个结点的树有 4 种染法；
2. 发现乘法 → $g(u) = (g(l)+1)(g(r)+1)$ ，不存在的孩子 $g = 0$ ；
3. 整齐的树先算好 → 满二叉树 $f[h] = (f[h-1]+1)^2$ ，预先打表；
4. 看清完全二叉树 → 只有最右那条路不整齐，路两旁全是满二叉树，第 j 步贡献 $f[j+k_j]$ ；
5. 二进制串当地图 → 把 s 减 1 就得到路径说明书，位 1 看左（完整），位 0 看右（矮一层）；
6. 边走边合并 → $ans \leftarrow (ans+1)(k+1)$ ，初值 1；
7. 写成代码 → 每层一次乘法， $O(\text{dep})$ 通过。

留给你思考

如果题目改成“不要求包含根”，答案该怎么算？

注意：这~~不等于~~在原答案上加 1（“再加个空集”是错的直觉）。原答案 $f[h]$ 统计的是“含根的块”，而“不含根的块”还有很多，两者数量级完全不同。正确的思路是换一个 DP 定义：记 $h(u)$ = 以 u 为**最高点**（块中最浅的那个点）的连通块数，它与第 2 关同形： $h(u) = (h(l)+1)(h(r)+1)$ 。于是全树的连通块总数 = $1 + \sum_u h(u)$ （那个 1 是空集；若题目不允许空集就去掉）。

在满二叉树上，这个总数记作 $A(h)$ ，可以推出

$$A(h) = 2 \cdot A(h-1) + f(h) - 1, \quad A(1) = 2$$

前几项是 **2, 7, 38, 751, ...**。你可以先用那棵 3 个结点的树手数验证：空集 1 + 单点 3 + 两点块 2（{根, 左}、{根, 右}）+ 三点块 1 = 7 ✓，与 $A(2) = 7$ 一致。再想想一般的完全二叉树要怎么做。